

Reviewer Pack — Minimal Tropical Discriminant for k-CLIQUE Monotone Circuit ...

Entry d5a25b74d4ce · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 04:37:52 UTC

Minimal Tropical Discriminant for k-CLIQUE Monotone Circuit Lower Bounds

Entry ID: d5a25b74d4ce **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 04:37:52 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For any DNF formula f representing the k-CLIQUE problem in monotone circuits, the minimal tropical discriminant (computed over the tropical semiring) of the characteristic polynomial of f is $\Omega(n^{\{3/2\}})$.’, ‘The minimal tropical discriminant refers to the smallest non-zero value in the tropical semiring obtained from the resultant of the characteristic polynomial with respect to a generic linear form.’, ‘For any DNF formula of size $\text{poly}(n)$, this minimal tropical discriminant is $O(\log n)$.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework for studying arithmetic properties of polynomials over the max-plus semiring, which can be used to analyze the complexity of monotone circuits.’, ‘By considering the characteristic polynomial and its tropical discriminant, we can gain insights into the structure of the DNF formulas representing computational problems like k-CLIQUE.’, ‘If the conjecture holds, it would suggest that there is a non-trivial arithmetic property of the polynomials associated with k-CLIQUE, which could potentially lead to lower bounds on the size of monotone circuits.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9bfb79f46fb486fb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the minimal tropical discriminants from at least 80% of the generated DNF formulas is greater than or equal to $n^{(3/2)}$ with a standard deviation less than 10% of $n^{(3/2)}$, and no single seed’s discriminant value exceeds $1.5 \cdot$

$n^{(3/2)}$. The conjecture is falsified if any seed produces a minimal tropical discriminant less than $n^{(3/2)}$ or a mean discriminant below $n^{(3/2)}$ with at least 80% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "monotone circuit complexity for k-CLIQUE" - "minimal tropical discriminant" AND characteristic polynomial" - "DNF formula" AND monotone circuits AND $\Omega(n^{\{3/2\}})$

Top relevant hits considered: - [<http://arxiv.org/abs/2509.14461v3>] Learning depth-3 circuits via quantum agnostic boosting - [<http://arxiv.org/abs/1311.1616v4>] Hardness Amplification and the Approximate Degree of Constant-Depth Circuits - [<http://arxiv.org/abs/2209.01584v1>] The Approximate Degree of DNF and CNF Formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_dnf_formula(n, k):
    if n < 1 or k < 1:
        return []

    variables = list(range(1, n + 1))
    clauses = []

    for _ in range(k):
        clause = random.sample(variables, random.randint(1, n))
        clauses.append(clause)

    return clauses

def characteristic_polynomial(dnf_formula):
    if not dnf_formula:
        return [1]

    n = len(dnf_formula[0])
```

```

poly = [1] * (n + 1)

for clause in dnf_formula:
    term = [1]
    for var in clause:
        term.append(-var)

    poly = polynomial_multiplication(poly, term)

return poly

def polynomial_multiplication(p1, p2):
    result = [0] * (len(p1) + len(p2) - 1)

    for i, coeff1 in enumerate(p1):
        for j, coeff2 in enumerate(p2):
            result[i + j] += coeff1 * coeff2

    return result

def tropical_discriminant(poly):
    if not poly:
        return None

    n = len(poly) - 1
    discriminant = float('-inf')

    for i in range(1, n):
        term = poly[i]
        discriminant = max(discriminant, term)

    return discriminant

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []

    for n in [5, 10, 15, 20, 30, 40]:
        k = random.randint(1, min(n // 2, 5))
        dnf_formula = generate_dnf_formula(n, k)

        if not dnf_formula:
            continue

        poly = characteristic_polynomial(dnf_formula)
        disc = tropical_discriminant(poly)

        if disc is None or disc <= 0:
            continue

        results.append((n, disc))

    if not results:
        return {
            "metric_name": "tropical_discriminant",
            "metric_value": float('nan'),

```

```

        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n_values = [n for n, _ in results]
disc_values = [disc for _, disc in results]

mean_disc = sum(disc_values) / len(disc_values)
std_disc = math.sqrt(sum((disc - mean_disc) ** 2 for disc in disc_values) / len(disc_values))

conjecture_holds = all(mean_disc >= n ** (3/2) for n, _ in results) and std_disc < 0.1 * mean_disc

return {
    "metric_name": "tropical_discriminant",
    "metric_value": mean_disc,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"mean<{mean_disc}, std<{std_disc}>"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_disc = sum(r['metric_value'] for r in results if not math.isnan(r['metric_value'])) / len(results)
    std_disc = math.sqrt(sum((r['metric_value'] - mean_disc) ** 2 for r in results if not math.isnan(r['metric_value'])) / len(results))

    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_disc} std={std_disc} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"{results[next(i for i, r in enumerate(results) if not r['conjecture_holds'])]}\"")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3ea597f.py", line 137, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b3ea597f.py", line 114, in run_trial
    dnf_formula = generate_dnf_formula(n, k)
    ^
```

NameError: name 'k' is not defined

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'k', which prevented the computation of the minimal tropical discriminants and thus the verification of the conjecture. | next: Define the variable 'k' in the test code and ensure all necessary variables are defined before running the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15046
2	preregistration	ollama_remote	glm4:latest	0	0	11090
3	novelty	ollama_remote	glm4:latest	0	0	9114
4	novelty	ollama_remote	glm4:latest	0	0	9146
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17646
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13427
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18147
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13158
9	judge	ollama_remote	glm4:latest	0	0	13241

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120015 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/d5a25b74d4ce.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d5a25b74d4ce.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d5a25b74d4ce.tar.gz` (if generated)